

- » Delving into the binary system
- » Digging into IP addresses
- » Finding out how subnetting works
- » Delving into ports
- » Looking at network address translation

Chapter 3

IP Addresses

One of the most basic components of TCP/IP is Internet Protocol (IP) addressing. Every device on a TCP/IP network must have a unique IP address. In this chapter, I describe the ins and outs of these IP addresses. Enjoy!

Understanding Binary

Before you can understand the details of how IP addressing works, you need to understand how the binary numbering system works because binary is the basis of IP addressing. If you already understand binary, please skip to the section “Introducing IP Addresses.” I don’t want to bore you with stuff that’s too basic.

Counting by ones

Binary is a counting system that uses only two numerals: 0 and 1. In the decimal system (with which most people are accustomed), you use ten numerals: 0–9. In an ordinary decimal number — such as 3,482 — the rightmost digit represents ones; the next digit to the left, tens; the next, hundreds; the next, thousands; and so on. These digits represent powers of ten: first 10^0 (which is 1); next, 10^1 (10); then 10^2 (100); then 10^3 (1,000); and so on.

In binary, you have only two numerals rather than ten, which is why binary numbers look somewhat monotonous, as in 110011, 101111, and 100001.

The positions in a binary number (called *bits* rather than *digits*) represent powers of two rather than powers of ten: 1, 2, 4, 8, 16, 32, and so on. To figure the decimal value of a binary number, you multiply each bit by its corresponding power of two and then add the results. The decimal value of binary 10111, for example, is calculated as follows:

$$1 \times 2^0 = 1 \times 1 = 1$$

$$1 \times 2^1 = 1 \times 2 = 2$$

$$1 \times 2^2 = 1 \times 4 = 4$$

$$0 \times 2^3 = 0 \times 8 = 0$$

$$1 \times 2^4 = 1 \times 16 = 16$$

$$\text{Total} = 1 + 2 + 4 + 0 + 16 = 23$$

Fortunately, converting a number between binary and decimal is something a computer is good at — so good, in fact, that you’re unlikely ever to need to do any conversions yourself. The point of learning binary is not to be able to look at a number such as 1110110110110 and say instantly, “Ah! Decimal 7,606!” (If you could do that, Piers Morgan would probably interview you, and they would even make a movie about you.)

Instead, the point is to have a basic understanding of how computers store information and — most important — to understand how the binary counting system works, which I describe in the following section.

Here are some of the more interesting characteristics of binary and how the system is similar to and differs from the decimal system:

» **In decimal, the number of decimal places allotted for a number determines how large the number can be.** If you allot six digits, for example, the largest number possible is 999,999. Because 0 is itself a number, however, a six-digit number can have any of 1 million different values.

Similarly, the number of bits allotted for a binary number determines how large that number can be. If you allot eight bits, the largest value that number can store is 11111111, which happens to be 255 in decimal.



TIP

» To quickly figure how many different values you can store in a binary number of a given length, use the number of bits as an exponent of two.

An eight-bit binary number, for example, can hold 2^8 values. Because 2^8 is 256, an eight-bit number can have any of 256 different values. This is why a *byte* — eight bits — can have 256 different values.

» This “powers of two” thing is why computers don’t use nice, even, round numbers in measuring such values as memory or disk space.

A value of 1K, for example, is not an even 1,000 bytes: It’s actually 1,024 bytes because 1,024 is 2^{10} . Similarly, 1MB is not an even 1,000,000 bytes but instead 1,048,576 bytes, which happens to be 2^{20} .



REMEMBER

One basic test of computer nerddom is knowing your powers of two because they play such an important role in binary numbers. Just for the fun of it, but not because you really need to know, Table 3-1 lists the powers of two up to 32.

TABLE 3-1 Powers of Two

Power	Bytes	Kilobytes	Power	Bytes	K, MB, or GB
2^1	2		2^{17}	131,072	128K
2^2	4		2^{18}	262,144	256K
2^3	8		2^{19}	524,288	512K
2^4	16		2^{20}	1,048,576	1MB
2^5	32		2^{21}	2,097,152	2MB
2^6	64		2^{22}	4,194,304	4MB
2^7	128		2^{23}	8,388,608	8MB
2^8	256		2^{24}	16,777,216	16MB
2^9	512		2^{25}	33,554,432	32MB
2^{10}	1,024	1K	2^{26}	67,108,864	64MB
2^{11}	2,048	2K	2^{27}	134,217,728	128MB
2^{12}	4,096	4K	2^{28}	268,435,456	256MB
2^{13}	8,192	8K	2^{29}	536,870,912	512MB
2^{14}	16,384	16K	2^{30}	1,073,741,824	1GB
2^{15}	32,768	32K	2^{31}	2,147,483,648	2GB
2^{16}	65,536	64K	2^{32}	4,294,967,296	4GB

Table 3-1 also shows the common shorthand notation for various powers of two. The abbreviation *K* represents 2^{10} (1,024). The *M* in *MB* stands for 2^{20} , or 1,024*K*, and the *G* in *GB* represents 2^{30} , which is 1,024*MB*. These shorthand notations don't have anything to do with TCP/IP, but they're commonly used for measuring computer disk and memory capacities, so I thought I'd throw them in at no charge because the table had extra room.

Doing the logic thing

One of the great things about binary is that it's very efficient at handling special operations: namely, logical operations. Four basic logical operations exist although additional operations are derived from the basic four operations. Three of the operations — AND, OR, and XOR — compare two binary digits (bits). The fourth (NOT) works on just a single bit.

The following list summarizes the basic logical operations:

- » **AND:** Compares two binary values. If both values are 1, the result of the AND operation is 1. If one or both of the values are 0, the result is 0.
- » **OR:** Compares two binary values. If at least one value is 1, the result of the OR operation is 1. If both values are 0, the result is 0.
- » **XOR:** Compares two binary values. If one of them is 1, the result is 1. If both values are 0 or if both values are 1, the result is 0.
- » **NOT:** Doesn't compare two values but simply changes the value of a single binary value. If the original value is 1, NOT returns 0. If the original value is 0, NOT returns 1.

Table 3-2 summarizes how AND, OR, and XOR work.

TABLE 3-2

Logical Operations for Binary Values

First Value	Second Value	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Logical operations are applied to binary numbers that have more than one binary digit by applying the operation one bit at a time. The easiest way to do this manually is to line the two binary numbers on top of one another and then write the result of the operation beneath each binary digit. The following example shows how you would calculate 10010100 AND 11011101:

```
10010100
AND 11011101
10010100
```

As you can see, the result is 10010100.

Working with the binary Windows Calculator

The Calculator program that comes with all versions of Windows has a special Programmer mode that many users don't know about. When you flip the Calculator into this mode, you can do instant binary and decimal conversions, which can occasionally come in handy when you're working with IP addresses.

To launch the Calculator, press the Windows key, type **Calculator** into the search bar, and then press Enter. When the Calculator is open, you can switch to Programmer mode by clicking the menu icon in the upper-left corner of the Calculator and choosing Programmer. In Programmer mode, you can do calculations in decimal (base 10), hexadecimal (base 16), octal (base 8), or binary (base 2). Figure 3-1 shows the Programmer mode for the latest version of the Calculator (as of this writing).

In the middle left of the Calculator window, you can see the current value displayed by the Calculator in hexadecimal (HEX), decimal (DEC), octal (OCT), and binary (BIN). You can also tell which base the main display shows, because it's highlighted with a bar. In Figure 3-1, the current mode is DEC, so the decimal value 100 is shown in large text in the upper-middle part of the display. You can switch the main display to hexadecimal, octal, or binary by clicking HEX, OCT, or BIN, respectively.

You can also see the current value in all four bases. Thus, in the figure, you can see that decimal 100 is 64 in hexadecimal, 144 in octal, and 01100100 in binary.

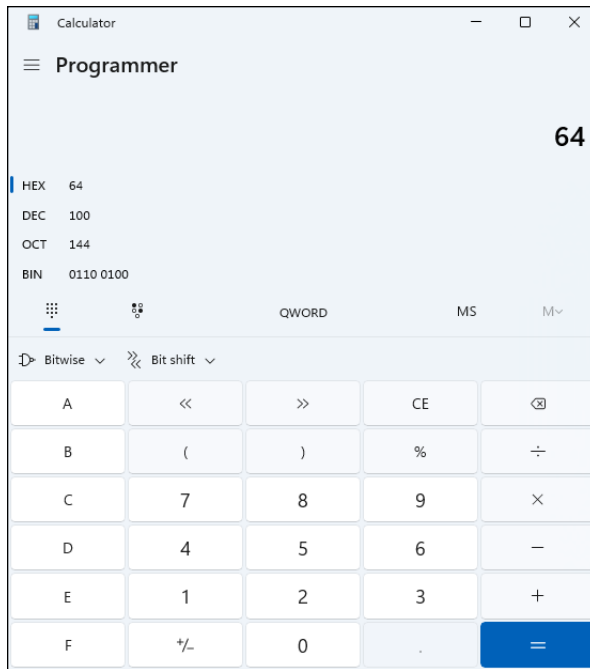


FIGURE 3-1:
The Windows
Calculator in Pro-
grammer mode.

Here are a few other things to note about the Programmer mode of the Calculator:

- » Although you can convert decimal values to binary values with the programmer Calculator, the Calculator can't handle the dotted-decimal IP address format that's described later in this chapter. To convert a dotted-decimal address to binary, just convert each octet separately. For example, to convert 172.65.48.120 to binary, first convert 172; then convert 65; then convert 48; and finally, convert 120.
- » The Programmer Calculator has several features that are designed specifically for binary operations, such as AND, OR, XOR, and so on.
- » The Programmer Calculator also has many other cool features for working with binary values. Spend some time exploring it when you have a few minutes!

Introducing IP Addresses

An *IP address* is a number that uniquely identifies every host on an IP network. IP addresses operate at the network layer of the TCP/IP protocol stack, so they are independent of lower-level data link layer MAC addresses, such as Ethernet MAC addresses.

IP addresses are 32-bit binary numbers, which means that theoretically, a maximum of something in the neighborhood of 4 billion unique host addresses can exist throughout the internet. You'd think that would be enough, but TCP/IP places certain restrictions on how IP addresses are allocated. These restrictions severely limit the total number of usable IP addresses. Many experts predict that we will run out of IP addresses soon. However, new techniques for working with IP addresses have helped to alleviate this problem, and a standard for 128-bit IP addresses has been adopted, though it still is not yet in widespread use.



TECHNICAL
STUFF

32-bit IP addresses are technically known as IP4 addresses, because they reflect version 4 of the IP. A newer version of IP, known as IP6, uses 128-bit addresses. For more information about IP6, see the sidebar “What about IP6?”

Networks and hosts

The Internet Protocol's primary purpose is to enable communications between networks. As a result, a 32-bit IP address actually consists of two parts:

- » **The network ID (or network address):** Identifies the network on which a host computer can be found
- » **The host ID (or host address):** Identifies a specific device on the network indicated by the network ID

Most of the complexity of working with IP addresses has to do with figuring out which part of the complete 32-bit IP address is the network ID and which part is the host ID, as described in the following sections.



TECHNICAL
STUFF

As I describe the details of how host IDs are assigned, you may notice that two host addresses seem to be unaccounted for. For example, the Class C addressing scheme, which uses eight bits for the host ID, allows only 254 hosts — not the 256 hosts you'd expect. The host ID can't be 0 (the host ID is all zeros) because that address is always reserved to represent the network itself. And the host ID can't be 255 (the host ID is all ones) because that host ID is reserved for use as a broadcast request that's intended for all hosts on the network.

The dotted-decimal dance

IP addresses are usually represented in a format known as *dotted-decimal notation*. In dotted-decimal notation, each group of eight bits — an *octet* — is represented by its decimal equivalent. For example, consider the following binary IP address:

```
11000000101010001000100000011100
```

To convert this value to dotted-decimal notation, first divide it into four octets, as follows:

```
11000000 10101000 10001000 00011100
```

Then, convert each of the octets to its decimal equivalent:

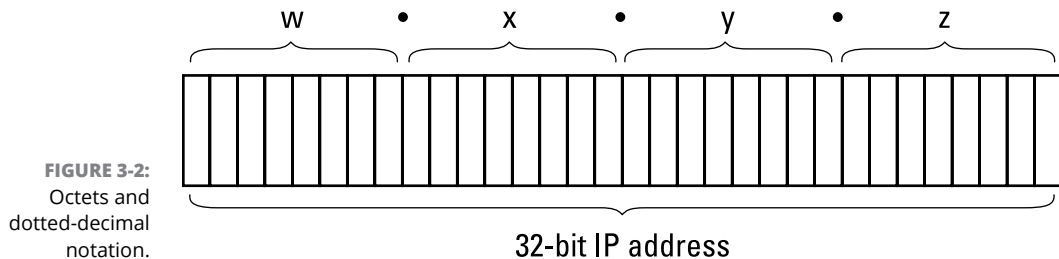
```
11000000 10101000 10001000 00011100
192      168      136      28
```

Then, use periods to separate the four decimal numbers, like this:

```
192.168.136.28
```

This is the format in which you'll usually see IP addresses represented.

Figure 3-2 shows how the 32 bits of an IP address are broken down into four octets of eight bits each. As you can see, the four octets of an IP address are often referred to as *w*, *x*, *y*, and *z*.



Classifying IP Addresses

When the original designers of the IP protocol created the IP addressing scheme, they could have assigned an arbitrary number of IP address bits for the network ID. The remaining bits would then be used for the host ID. For example, suppose that the designers decided that half of the address (16 bits) would be used for the network, and the remaining 16 bits would be used for the host ID. The result of that scheme would be that the internet could have a total of 65,536 networks, and each of those networks could have 65,536 hosts.

In the early days of the internet, this scheme probably seemed like several orders of magnitude more than would ever be needed. However, the IP designers realized from the start that few networks would actually have tens of thousands of hosts. Suppose that a network of 1,000 computers joins the internet and is assigned one of these hypothetical network IDs. Because that network will use only 1,000 of its 65,536 host addresses, more than 64,000 IP addresses would be wasted.

WHAT ABOUT IPV6?

Most of the current internet is based on version 4 of the Internet Protocol, also known as IPv4. IPv4 has served the internet well for more than 40 years. However, the growth of the internet has put a lot of pressure on IPv4's limited 32-bit address space. This chapter describes how IPv4 has evolved to make the best possible use of 32-bit addresses. Eventually, though, all the addresses will be assigned, and the IPv4 address space will be filled to capacity. When that happens, the internet will have to migrate to the next version of IP, known as IPv6.

IPv6 is also called *IP next generation*, or *IPng*, in honor of the favorite television show of most internet gurus, *Star Trek: The Next Generation*.

IPv6 offers several advantages over IPv4, but the most important is that it uses 128 bits for internet addresses instead of 32 bits. The number of host addresses possible with 128 bits is a number so large that it would have made Carl Sagan proud. It doesn't just double or triple the number of available addresses, or even a thousand-fold or even a million-fold. Just for the fun of it, here is the number of unique internet addresses provided by IPv6:

340,282,366,920,938,463,463,374,607,431,768,211,456

This number is so large it defies understanding. If the Internet Assigned Numbers Authority (IANA) had been around at the creation of the universe and started handing out IPv6 addresses at a rate of one per millisecond — that is, 1,000 addresses every second — it would now, 15 billion years later, have not yet allocated even 1 percent of the available addresses.

The transition from IPv4 to IPv6 has been slow. IPv6 is available on all new computers and has been supported on Windows since Windows XP Service Pack 1 (released in 2002). However, most ISPs still base their service on IPv4. Thus, the internet will continue to be driven by IPv4 for at least a few more years.

(continued)

(continued)

Note: This is now the eighth edition of this book. Every previous edition of this book, all the way back to the very first edition published in 2004, has had this very sidebar. In 2004, I said that the internet would continue to be driven by IPv4 for at least a few more years. “A few more years” has morphed into 14 years, and we’re still living in the world of IPv4. Make no mistake: The world will eventually run out of IPv4 addresses, and we’ll have to migrate to IPv6 . . . in a few years, whatever that means.

As a solution to this problem, the idea of IP address classes was introduced. The IP protocol defines five different address classes: A, B, C, D, and E. Each of the first three classes, A–C, uses a different size for the network ID and host ID portion of the address. Class D is for a special type of address called a *multicast address*. Class E is an experimental address class that isn’t used.

The first four bits of the IP address are used to determine into which class a particular address fits, as follows:

- » **Class A:** The first bit is zero.
- » **Class B:** The first bit is one, and the second bit is zero.
- » **Class C:** The first two bits are both one, and the third bit is zero.
- » **Class D:** The first three bits are all one, and the fourth bit is zero.
- » **Class E:** The first four bits are all one.

Because Class D and E addresses are reserved for special purposes, I focus the rest of the discussion here on Class A, B, and C addresses. Table 3–3 summarizes the details of each address class.

TABLE 3-3 IP Address Classes

Class	Address Number Range	Starting Bits	Length of Network ID	Number of Networks	Hosts
A	1–126.x.y.z	0	8	126	16,777,214
B	128–191.x.y.z	10	16	16,384	65,534
C	192–223.x.y.z	110	24	2,097,152	254

Class A addresses

Class A addresses are designed for very large networks. In a Class A address, the first octet of the address is the network ID, and the remaining three octets are the host ID. Because only eight bits are allocated to the network ID and the first of these bits is used to indicate that the address is a Class A address, only 126 Class A networks can exist in the entire internet. However, each Class A network can accommodate more than 16 million hosts.

Only about 40 Class A addresses are actually assigned to companies or organizations. The rest are either reserved for use by the Internet Assigned Numbers Authority (IANA) or are assigned to organizations that manage IP assignments for geographic regions such as Europe, Asia, and Latin America.

In case you're interested, you can find a complete list of all the Class A address assignments at www.iana.org/assignments/ipv4-address-space/ipv4-address-space.xml.

You may have noticed in Table 3-3 that Class A addresses end with 126.x.y.z, and Class B addresses begin with 128.x.y.z. What happened to 127.x.y.z? This special range of addresses is reserved for loop-back testing, so these addresses aren't assigned to public networks.



TIP

The special address 127.0.0.1 is called the *loop-back address*. A device at any IP address that sends a message to 127.0.0.1 is sending a message to itself. This may sound useless, but it actually plays an important role in troubleshooting network problems.

Class B addresses

In a Class B address, the first two octets of the IP address are used as the network ID, and the second two octets are used as the host ID. Thus, a Class B address comes close to my hypothetical scheme of splitting the address down the middle, using half for the network ID and half for the host ID. It isn't identical to this scheme, however, because the first two bits of the first octet are required to be 10, in order to indicate that the address is a Class B address. As a result, a total of 16,384 Class B networks can exist. All Class B addresses fall within the range 128.x.y.z to 191.x.y.z. Each Class B address can accommodate more than 65,000 hosts.

The problem with Class B networks is that even though they are much smaller than Class A networks, they still allocate far too many host IDs. Very few networks have tens of thousands of hosts. Thus, careless assignment of Class B addresses can lead to a large percentage of the available host addresses being wasted on organizations that don't need them.

Class C addresses

In a Class C address, the first three octets are used for the network ID, and the fourth octet is used for the host ID. With only eight bits for the host ID, each Class C network can accommodate only 254 hosts. However, with 24 network ID bits, Class C addresses allow for more than 2 million networks.

The problem with Class C networks is that they're too small. Although few organizations need the tens of thousands of host addresses provided by a Class B address, many organizations need more than a few hundred. The large discrepancy between Class B networks and Class C networks is what led to the development of *subnetting*, which I describe in the next section.

Subnetting

Subnetting is a technique that lets network administrators use the 32 bits available in an IP address more efficiently by creating networks that aren't limited to the scales provided by Class A, B, and C IP addresses. With subnetting, you can create networks with more realistic host limits.

Subnetting provides a more flexible way to designate which portion of an IP address represents the network ID and which portion represents the host ID. With standard IP address classes, only three possible network ID sizes exist: 8 bits for Class A, 16 bits for Class B, and 24 bits for Class C. Subnetting lets you select an arbitrary number of bits to use for the network ID.

Two reasons compel people to use subnetting. The first is to allocate the limited IP address space more efficiently. If the internet were limited to Class A, B, or C addresses, every network would be allocated 254, 64,000, or 16 million IP addresses for host devices. Although many networks with more than 254 devices exist, few (if any) exist with 64,000, let alone 16 million. Unfortunately, any network with more than 254 devices would need a Class B allocation and probably waste tens of thousands of IP addresses.

The second reason for subnetting is that even if a single organization has thousands of network devices, operating all those devices with the same network ID would slow the network to a crawl. The way TCP/IP works dictates that all the computers with the same network ID must be on the same physical network. The physical network comprises a single *broadcast domain*, which means that a single network medium must carry all the traffic for the network. For performance reasons, networks are usually segmented into broadcast domains that are smaller than even Class C addresses provide.

Subnets

A *subnet* is a network that falls within a Class A, B, or C network. Subnets are created by using one or more of the Class A, B, or C host bits to extend the network ID. Thus, instead of the standard 8-, 16-, or 24-bit network ID, subnets can have network IDs of any length.

Figure 3-3 shows an example of a network before and after subnetting has been applied. In the unsubnetted network, the network has been assigned the Class B address 144.28.0.0. All the devices on this network must share the same broadcast domain.

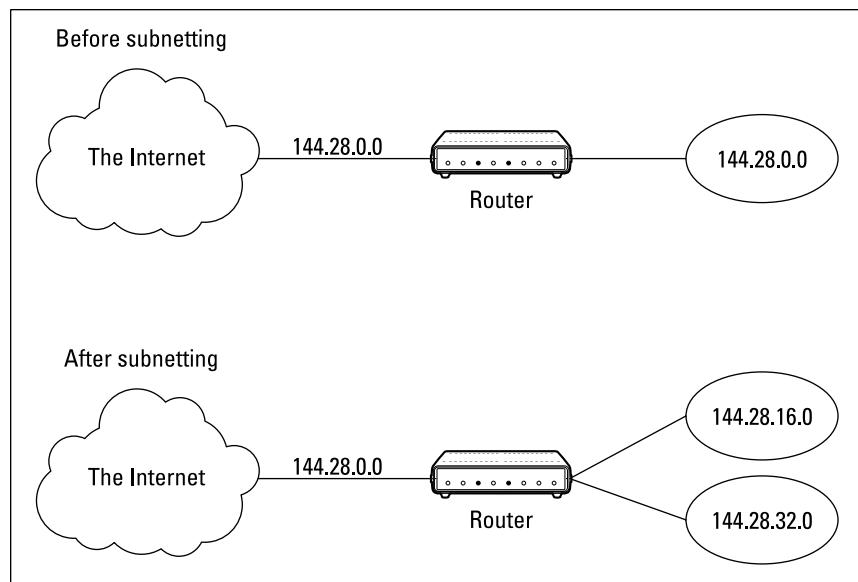


FIGURE 3-3:
A network
before and after
subnetting.

In the second network, the first four bits of the host ID are used to divide the network into two small networks, identified as subnets 16 and 32. To the outside world (that is, on the other side of the router), these two networks still appear to be a single network identified as 144.28.0.0. For example, the outside world considers the device at 144.28.16.22 to belong to the 144.28.0.0 network. As a result, a packet sent to this device will be delivered to the router at 144.28.0.0. The router then considers the subnet portion of the host ID to decide whether to route the packet to subnet 16 or subnet 32.

Subnet masks

For subnetting to work, the router must be told which portion of the host ID should be used for the subnet network ID. This little sleight of hand is accomplished by using another 32-bit number, known as a *subnet mask*. Those IP address bits that represent the network ID are represented by a 1 in the mask, and those bits that represent the host ID appear as a 0 in the mask. As a result, a subnet mask always has a consecutive string of ones on the left, followed by a string of zeros.

For example, the subnet mask for the subnet shown in Figure 3-3, where the network ID consists of the 16-bit network ID plus an additional 4-bit subnet ID, would look like this:

```
11111111 11111111 11110000 00000000
```

In other words, the first 20 bits are ones, and the remaining 12 bits are zeros. Thus, the complete network ID is 20 bits in length, and the actual host ID portion of the subnetted address is 12 bits in length.

To determine the network ID of an IP address, the router must have both the IP address and the subnet mask. The router then performs a bitwise operation called a *logical AND* on the IP address in order to extract the network ID. To perform a logical AND, each bit in the IP address is compared with the corresponding bit in the subnet mask. If both bits are 1, the resulting bit in the network ID is set to 1. If either of the bits are 0, the resulting bit is set to 0.

For example, here's how the network address is extracted from an IP address using the 20-bit subnet mask from the previous example:

```
IP address: 10010000 00011100 00010000 00010001 (144.28.16.17)
Subnet mask: 11111111 11111111 11110000 00000000
Network ID: 10010000 00011100 00010000 00000000 (144.28.16.0)
```

Thus, the network ID for this subnet is 144.28.16.0.

The subnet mask itself is usually represented in dotted-decimal notation. As a result, the 20-bit subnet mask used in the previous example would be represented as 255.255.240.0:

```
Subnet mask: 11111111 11111111 11110000 00000000
255 . 255 . 240 . 0
```

Don't confuse a subnet mask with an IP address. A subnet mask doesn't represent any device or network on the internet. It's just a way of indicating which portion of an IP address should be used to determine the network ID.

Note that a subnet mask cannot be an arbitrary collection of octets. Instead, a subnet mask always has a certain number of binary 1s on its left side, and the remaining bits of the mask are always 0. This limits the dotted-decimal representation of a subnet mask to certain values.



TIP

You can spot a subnet mask right away because the first octet is always 255, and 255 is not a valid first octet for any class of IP address.

Network prefix notation

Because a subnet mask always begins with a consecutive sequence of ones to indicate which bits to use for the network ID, you can use a shorthand notation — a *network prefix* — to indicate how many bits of an IP address represent the network ID. The network prefix is indicated with a slash immediately after the IP address, followed by the number of network ID bits to use. For example, the IP address 144.28.16.17 with the subnet mask 255.255.240.0 can be represented as 144.28.16.17/20 because the subnet mask 255.255.240.0 has 20 network ID bits.

Network prefix notation is also called *classless interdomain routing* notation (CIDR, for short) because it provides a way of indicating which portion of an address is the network ID and which is the host ID without relying on standard address classes.

Default subnets

The *default subnet masks* are three subnet masks that correspond to the standard Class A, B, and C address assignments. These default masks are summarized in Table 3-4.

TABLE 3-4 The Default Subnet Masks

Class	Binary	Dotted-Decimal	Network Prefix
A	11111111 00000000 00000000 00000000	255.0.0.0	/8
B	11111111 11111111 00000000 00000000	255.255.0.0	/16
C	11111111 11111111 11111111 00000000	255.255.255.0	/24



TIP

Keep in mind that a subnet mask is not actually required to use one of these defaults because the IP address class can be determined by examining the first three bits of the IP address. If the first bit is 0, the address is Class A, and the subnet mask 255.0.0 is applied. If the first two bits are 10, the address is Class B,

and 255.255.0.0 is used. If the first three bits are 110, the Class C default mask 255.255.255.0 is used.

The great subnet roundup

You should know about a few additional restrictions that are placed on subnets and subnet masks. In particular

- » **The minimum number of network ID bits is eight.** As a result, the first octet of a subnet mask is always 255.
- » **The maximum number of network ID bits is 30.** You have to leave at least two bits for the host ID portion of the address to allow for at least two hosts. If you use all 32 bits for the network ID, that leaves no bits for the host ID. Obviously, that won't work. Leaving just one bit for the host ID won't work, either, because a host ID of all ones is reserved for a broadcast address, and all zeros refers to the network itself. Thus, if you use 31 bits for the network ID and leave only 1 for the host ID, host ID 1 would be used for the broadcast address, and host ID 0 would be the network itself, leaving no room for actual hosts. That's why the maximum network ID size is 30 bits.
- » **Because the network ID portion of a subnet mask is always composed of consecutive bits set to 1, only eight values are possible for each octet of a subnet mask:** 0, 128, 192, 224, 248, 252, 254, and 255.
- » **A subnet address can't be all zeros or all ones.** Thus, the number of unique subnet addresses is two less than two raised to the number of subnet address bits. For example, with three subnet address bits, six unique subnet addresses are possible ($2^3 - 2 = 6$). This implies that you must have at least two subnet bits. (If a single-bit subnet mask were allowed, it would violate the "can't be all zeros or all ones" rule because the only two allowed values would be 0 or 1.)

SUBNETS VERSUS VLANS

All of this talk of subnets might have you wondering: What's the difference between subnets and virtual local area networks (VLANs)? If you've read Book 1, Chapter 2, you know that VLANs are a divide-and-conquer technique for managing large networks. Subnetting is also a divide-and-conquer technique.

So, are they the same thing, and do they serve the same purpose?

The answer is: No, but sort of kind of. But really, no.

Although VLANs and subnets seem similar, VLANs are a layer 2 construct, and subnets are a layer 3 construct.

In other words, VLANs have nothing to do with IP addresses and subnets have nothing to do with MAC addresses.

That being said, it is very common — and usually desirable — to design your network with a one-to-one correspondence between VLANs and IP subnets. This usually simplifies the task of managing both.

As an example, suppose you want to divide a single-office network with just a hundred or fewer users into three groups: End-user devices like computers and printers, servers and network devices, and Voice over Internet Protocol (VoIP) phones. You could use three VLANs to do this — call them VLAN 10, VLAN 20, and VLAN 30.

You could then use three subnets — 192.168.10.x, 192.168.20.x, and 192.168.30.x.

There's a natural correspondence between these three VLANs and the three subnets, and network setup and management will be easier because the VLANs and subnets correspond to one another.

Although you can have a single VLAN that supports multiple subnets, in most networks there is a one-to-one correspondence that allows the benefits of VLANs and subnets to complement one another.

IP block parties

A subnet can be thought of as a range or block of IP addresses that have a common network ID. For example, the CIDR 192.168.1.0/28 represents the following block of 14 IP addresses:

192.168.1.1	192.168.1.2	192.168.1.3	192.168.1.4
192.168.1.5	192.168.1.6	192.168.1.7	192.168.1.8
192.168.1.9	192.168.1.10	192.168.1.11	192.168.1.12
192.168.1.13	192.168.1.14		

Given an IP address in CIDR notation, it's useful to be able to determine the range of actual IP addresses that the CIDR represents. This matter is straightforward when the octet within which the network ID mask ends happens to be 0, as in the preceding example. You just determine how many host IDs are allowed based on the size of the network ID and count them off.

However, what if the octet where the network ID mask ends is not 0? For example, what are the valid IP addresses for 192.168.1.100 when the subnet mask is 255.255.255.240? In that case, the calculation is a little harder. The first step is to determine the actual network ID. You can do that by converting both the IP address and the subnet mask to binary and then extracting the network ID as in this example:

```
IP address:  11000000 10101000 00000001 01100100 (192.168.1.100)
Subnet mask: 11111111 11111111 11111111 11110000
Network ID:  11000000 10101000 00000001 01100000 (192.168.1.96)
```

As a result, the network ID is 192.168.1.96.

Next, determine the number of allowable hosts in the subnet based on the network prefix. You can calculate this by subtracting the last octet of the subnet mask from 254. In this case, the number of allowable hosts is 14.

To determine the first IP address in the block, add 1 to the network ID. Thus, the first IP address in my example is 192.168.1.97. To determine the last IP address in the block, add the number of hosts to the network ID. In my example, the last IP address is 192.168.1.110. As a result, the 192.168.1.100 with subnet mask 255.255.255.240 designates the following block of IP addresses:

```
192.168.1.97 192.168.1.98 192.168.1.99 192.168.1.100
192.168.1.101 192.168.1.102 192.168.1.103 192.168.1.104
192.168.1.105 192.168.1.106 192.168.1.107 192.168.1.108
192.168.1.109 192.168.1.110
```

Private and public addresses

Any host with a direct connection to the internet must have a globally unique IP address. However, not all hosts are connected directly to the internet. Some are on networks that aren't connected to the internet. Some hosts are hidden behind firewalls, so their internet connection is indirect.

Several blocks of IP addresses are set aside just for this purpose, for use on private networks that are not connected to the internet or to use on networks that are hidden behind a firewall. Three such ranges of addresses exist, summarized in Table 3-5. Whenever you create a private TCP/IP network, you should use IP addresses from one of these ranges.

TABLE 3-5

Private Address Spaces

CIDR	Subnet Mask	Address Range
10.0.0.0/8	255.0.0.0	10.0.0.1–10.255.255.254
172.16.0.0/12	255.240.0.0	172.16.1.1–172.31.255.254
192.168.0.0/16	255.255.0.0	192.168.0.1–192.168.255.254

Pondering Ports

When you use an IP address, you often associate that IP address with a *port*, which enables a connection to a particular service. The best-known port is port 80, which corresponds to the HTTP of the World Wide Web. The combination of a transport protocol (for example, TCP), an IP address, and a port is called an *internet socket*.

Although IP addresses are defined at layer 3 of the OSI model (the network layer), ports are a layer 4 construct. Layer 4 is the transport layer, so it makes sense that ports would live there.

Ports are commonly combined with IP addresses when used in URLs (also known as web addresses.) I dive deep into URLs in Book 2, Chapter 6, so hold tight.

Ports are represented by 32-bit numbers, so they range from 0 to 65535. There are three ranges of port numbers:

- » **0 to 1023:** These are called *well-known ports*, and they're used for the widely used services available on the internet.
- » **1024 to 49151:** These are called *registered ports*, and they're assigned by the internet's governing authorities to various service providers. For example, Apple's iTunes uses port 3689 and Adobe's Media Server uses port 8134.
- » **49152 to 65535:** These are called *dynamic ports*, *private ports*, or *ephemeral ports*. These ports cannot be registered and are used only for a specific communication.

Understanding Network Address Translation

Nearly all firewalls use a technique called *network address translation* (NAT) to hide the actual IP address of a computer on the local network from the outside world. When that's the case, the NAT device must use a globally unique IP address to

represent the computer to the internet. Behind the firewall, though, the computer has a private IP address. When packets cross the firewall, the NAT device translates the private IP address to the public IP address and vice versa.

NAT is one of the foundational techniques that enables the internet to work. It's the way an organization can have dozens, hundreds, or thousands of computers on its network without requiring a separate public IP address for each computer. Instead, each organization has a relatively small number of public IP addresses that are assigned to the public-facing interfaces of its firewall(s). NAT enables all the computers behind the firewall to communicate with the internet, piggybacking on the public IP address of the firewall itself.

Consider what typically happens when a user sends a request to a local HTTP server — that is, an HTTP server that is on the same network as the user. Let's assume that the IP address of the local HTTP server is 192.168.0.100, and the IP address of the user's computer is 192.168.0.50. What happens is this:

1. The user's computer sends an HTTP request in the form of an IP packet with the following address information:
 - For the source, the transport protocol is TCP, the IP address is 192.168.0.50. The port number for the source is chosen by client and is typically a high port number. For this example, I'll use port 45444 for the source port.
 - For the destination, the transport protocol is TCP, the IP address is 192.168.0.100, and the port is 80.
2. The HTTP server receives the request, processes it, and sends back an HTTP response in the form of an IP packet with the following address information:
 - For the source, the transport protocol is TCP, the IP address is 192.168.0.100, and the port is 80.
 - For the destination, the transport protocol is TCP, the IP address is 192.168.0.50, and the port is 45444 (the port that was chosen by the client).

This won't work if the user wants to send a request to an HTTP server on the internet, because the IP address of the user's computer is a private address, not a public address. So, the HTTP server won't be able to send a response to 192.168.0.50 because such an address doesn't exist on the public internet.

That's where NAT comes in.

The magic of NAT is handled by the firewall itself. The basic idea of NAT is that the firewall maintains an internal table of outgoing packets so it can remember which computer in the local network has requested information from sites on the

public internet. Because more than one computer may make requests for information from the same internet site, NAT exploits ephemeral ports (see the preceding section) to keep things straight.

Let me walk you through an example. But first, let's assume that the firewall in this example has the following IP addresses:

» **Outside IP address (public):** 75.68.10.201

» **Inside IP address (private):** 192.168.0.1

Let's also assume that the HTTP server is at 99.84.206.125 (which happens to be Wiley's web server), and a user whose private IP address is 192.168.0.50 uses a web browser to request information from the HTTP server. The HTTP request will have the following address information:

» **Source IP:** 192.168.0.50

» **Source port:** 45444

» **Destination IP:** 99.84.206.125

» **Destination port:** 80

Here's how it works:

1. The firewall sees this packet and realizes that it must substitute its own IP address (let's assume 192.168.0.1).
2. The firewall selects a random port number from a pool of ephemeral port numbers, which it will use to keep track of the request.
For example, let's say it picks port 42003.
3. The firewall records the following information in its NAT table for this request:
 - Source IP: 192.168.0.50
 - Source port: 45444
 - Destination IP: 99.84.206.125
 - Destination port: 80
 - Temporary port: 42003
4. The firewall modifies the packet by substituting its own public IP address for the source IP and the temporary port for the source port.

5. The firewall sends the modified packet to the public internet.
The modified packet contains the following information:
 - Source IP: 75.68.10.201
 - Source port: 42003
 - Destination IP: 99.84.206.125
 - Destination port: 80
6. A few seconds later, the firewall receives an incoming HTTP response message with the following address information:
 - Source IP: 99.84.206.125
 - Source port: 80
 - Destination IP: 75.68.10.201
 - Destination port: 42003
7. The firewall peruses its NAT table and finds that this response matches the entry it recorded in Step 3.
8. The firewall retrieves the original source IP address and port from the NAT table and substitutes it for the destination IP and port.
The modified response message now has the following address information:
 - Source IP: 99.84.206.125
 - Source port: 80
 - Destination IP: 192.168.0.50
 - Destination port: 45444
9. The firewall places the modified packet on the inside interface (that is, the local network), where the network can then deliver the packet to the original requestor (the user at 192.168.0.50).

So, what happens if two or more users have requests to the same web server at the same time? NAT is able to figure it out because each of those requests has a different temporary port number. For example, the user in the preceding example got port 42003. Another user sending a request to the same web server might get port 43859. When the reply comes back from the web server, NAT looks at the destination port to determine which local computer should receive the reply.